CSA4707-Deep Learning

Experiment 4: Overfitting Analysis

Aim:

To analyse the performance of overfitting using a chosen dataset with Python.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score

def true_fun(X):
    return np.cos(1.5 * np.pi * X)

np.random.seed(0)

n_samples = 30
degrees = [1, 4, 15]

X = np.sort(np.random.rand(n_samples))
y = true_fun(X) + np.random.randn(n_samples) * 0.1

plt.figure(figsize=(14, 5))

for i in range(len(degrees)):
    ax = plt.subplot(1, len(degrees), i + 1)
    plt.setp(ax, xticks=(), yticks=())

    polynomial_features = PolynomialFeatures(
        degree=degrees[i], include_bias=False
    )
    linear_regression = LinearRegression()

    pipeline = Pipeline([
        ("polynomial_features", polynomial_features),
        ("linear_regression", linear_regression)
    ])
```

```
    pipeline.fit(X[:, np.newaxis], y)

    scores = cross_val_score(
        pipeline, X[:, np.newaxis], y,
        scoring="neg_mean_squared_error", cv=10
    )

    X_test = np.linspace(0, 1, 100)

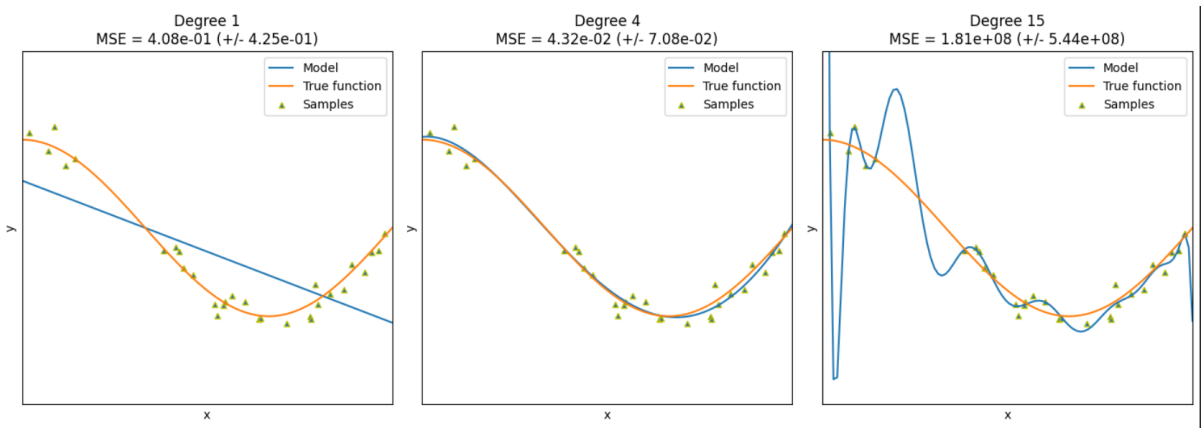
    plt.plot(
        X_test, pipeline.predict(X_test[:, np.newaxis]),
        label="Model"
    )
    plt.plot(X_test, true_fun(X_test), label="True function")
    plt.scatter(X, y, edgecolor="y", s=20, marker="^", label="Samples")

    plt.xlabel("x")
    plt.ylabel("y")
    plt.xlim((0, 1))
    plt.ylim((-2, 2))
    plt.legend(loc="best")

    plt.title(
        "Degree {}\nMSE = {:.2e} (+/- {:.2e})".format(
            degrees[i], -scores.mean(), scores.std()
        )
    )

plt.tight_layout()
plt.show()
```

Output:



Result:

The polynomial regression models of degrees 1, 4, and 15 were plotted successfully. The degree-15 model demonstrates overfitting by fitting the training data too closely, while the lower-degree models provide simpler fits.